*"Publish the fact. Consume it once. Never lose the message."*

MODULE 31

# Kafka Integration with Spring Boot

Learn how to build scalable, event-driven applications by integrating Apache Kafka with Spring Boot -- producers, consumers, message handling, error management, and best practices for reliable event processing.

TEKsystems Global Services

# WORKFORCE DEVELOPMENT

TEKsystems Global Services

# WORKFORCE DEVELOPMENT

# MODULE 31 OVERVIEW

*Integrate Apache Kafka with Spring Boot to publish, consume, and reliably process real-time customer events.*

Spring Kafka turns raw Kafka concepts into production Spring Boot code -- this module covers KafkaTemplate producers, @KafkaListener consumers, and the error handling and Dead Letter Topic patterns that keep event processing reliable.

## DURATION & LAB

### 1 Hour Theory

Spring Kafka components, KafkaTemplate producers, @KafkaListener consumers, and consumer groups.

### 2.5 Hours Lab

Northstar CRM Listeners: keyed publish, idempotent consume, Dead Letter Topics, and reliability proof.

### Scenario

Notifications must survive poison messages and replays without double-sending.

## MODULE ARC

### Publish the Facts

KafkaTemplate producers, keyed messages, and JSON serialization for CRM events.

### Consume Reliably

@KafkaListener, consumer groups, and idempotent processing that avoids double-handling.

### Build the Lab

Wire the Northstar CRM Listeners lab end to end with Dead Letter Topics and passing tests.

**KEY TAKEAWAY** Total time: 3.5 hours (1 hour theory + 2.5 hours hands-on lab). By the end, you will publish and consume Kafka events reliably from a real Spring Boot service.

# WHY SPRING KAFKA MATTERS

*Spring Kafka simplifies integration with Apache Kafka and empowers developers to build scalable, reliable, and maintainable event-driven applications.*

## WHAT SPRING KAFKA GIVES YOU

- **Seamless Integration —** native support and abstractions that make it easy to integrate Kafka with Spring Boot.
- **Developer Productivity —** high-level APIs like KafkaTemplate and @KafkaListener reduce boilerplate code.
- **Reliability & Robustness —** built-in features for error handling, retries, dead letter queues, and offset management.
- **Scalability —** easily build applications that scale horizontally to handle millions of events.
- **Enterprise Ready —** production-grade features, observability support, and strong community adoption.

## BUSINESS IMPACT

- **Real-Time Responsiveness —** process events instantly and respond to business changes as they happen.
- **Decoupled Applications —** reduce dependencies between services and increase flexibility.
- **Improved Reliability —** ensure message delivery and consistency with robust error handling.
- **Higher Throughput —** handle large volumes of events efficiently without impacting performance.
- **Lower Costs —** optimize resource usage and reduce operational overhead.

## SPRING KAFKA HELPS YOU FOCUS ON BUSINESS LOGIC

**Simple APIs** | **Out-of-the-Box Configuration** | **Flexible & Extensible** | **Monitoring & Observability** | **Large Community & Ecosystem**

**KEY TAKEAWAY** Spring Kafka bridges the power of Apache Kafka with the productivity of the Spring ecosystem -- helping you build modern, event-driven applications faster and with confidence.

# Why Spring Kafka Matters

Simplifying Event-Driven Architectures with the Power of Spring


## Business Impact

- Faster Time to Market
- Lower Operational Cost
- Scalable & Future Ready Architecture
- Higher System Reliability
- Better Customer Experience

# MODULE LEARNING OBJECTIVES

*By the end of this module, you will be able to design, build, and validate a complete Kafka integration in a Spring Boot application.*

### 1. Spring Kafka Fundamentals

Explain the role of Spring Kafka and its key components.

### 2. Integration Architecture

Understand message flow between producers, cluster, and consumers.

### 3. Kafka Producers

Build producers using KafkaTemplate with proper serialization.

### 4. Kafka Consumers

Implement consumers with @KafkaListener and consumer groups.

### 5. Advanced Messaging

Use message keys, offsets, error handling, and DLQs.

### 6. Reliable Processing

Achieve idempotency and at-least-once / exactly-once delivery.

### 7. Monitor and Optimize

Analyze performance and tune configurations for throughput.

### 8. Best Practices

Apply enterprise practices for topics, security, and scalability.

### 9. End-to-End Integration

Build a complete app that publishes and consumes events.

### 10. Hands-On Lab

Build, test, and validate Kafka integration in the graded lab.

## MODULE DURATION

**Theory: 1 Hour** | **Lab: 2.5 Hours** | **Total: 3.5 Hours**

**KEY TAKEAWAY** You will be equipped with the skills to seamlessly integrate Kafka with Spring Boot and build scalable, reliable, and event-driven applications ready for production.

# BUSINESS SCENARIO: EVENT-DRIVEN CUSTOMER NOTIFICATIONS

*Northstar CRM must notify downstream systems the moment a customer is created or changes status -- without losing events or double-processing replays.*

## THE PROBLEM

- Agents create and update customers through the REST API built in Modules 24-29 -- but notification and audit systems learn nothing unless they poll the database.
- Leadership now requires asynchronous notification: creating Amina or activating Ravi must trigger downstream processes in real time, not on a schedule.
- Duplicate delivery (rebalance, retry, replay) must never send duplicate SMS/email side effects, and an invalid contract must never retry forever.

## THE FIX: A VERSIONED CUSTOMEREVENT

- Publish a keyed CustomerEvent v1 after every successful customer write -- the event reflects a fact that already happened.
- Consume idempotently -- process each unique eventId exactly once for business-effect purposes, even if Kafka redelivers it.
- Recover poison messages (bad key, unsupported version) to a Dead Letter Topic after a bounded number of retries.

## FIXTURES USED THROUGHOUT THIS MODULE

| ID / Key | Value | Role | Notes |
|---|---|---|---|
| **CUS-1001** | Amina Khan | Primary publish/consume fixture | Status ACTIVE |
| **CUS-1002** | Ravi Singh | Second key / status-update fixture | Status PROSPECT |
| **crm.customer-events.v1** | Kafka topic | Primary event topic (from Lab 30) | Keyed by customerId |
| **lab-request-001** | correlationId | Traceability on every event and log line | Also used in Module 29 |

**KEY TAKEAWAY** This module builds the exact event backbone graded in Lab 31: KafkaTemplate publish, @KafkaListener consume, idempotent processing, and DLT recovery for Northstar CRM.

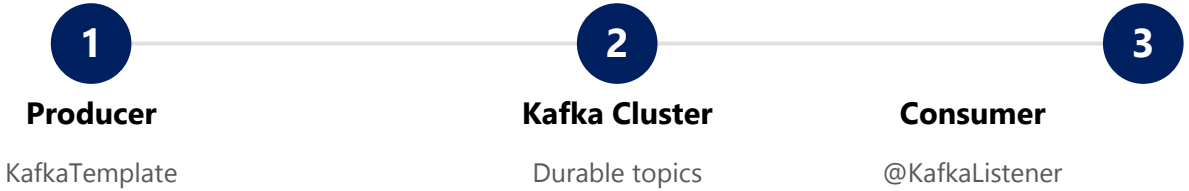
# INTRODUCTION TO SPRING KAFKA

*Spring Kafka is the official Spring project for building applications that use Apache Kafka for messaging and stream processing.*

## WHAT IS SPRING KAFKA?

- **Simplifies Development —** a Spring project that simplifies building Kafka-based messaging solutions.
- **High-Level Abstractions —** sits over the native Kafka client, reducing boilerplate code.
- **Deep Spring Integration —** integrates with Spring Boot, Spring Framework, and Spring Cloud.
- **Production Ready —** makes it easy to build scalable, reliable, event-driven applications.

## HOW SPRING KAFKA WORKS




Spring Kafka sits on top of the Kafka client library and provides convenient abstractions for producers and consumers, connected through a durable, replicated Apache Kafka cluster.

**Durable** **Scalable** **Fault-Tolerant** **High Throughput**

## WHY IT MATTERS

**Less Boilerplate** **Seamless Integration** **Faster Development**

**Production Ready**

**KEY TAKEAWAY** Spring Kafka provides developer-friendly APIs and integrations that make working with Apache Kafka in Spring applications productive and efficient.

# Introduction to Spring Kafka

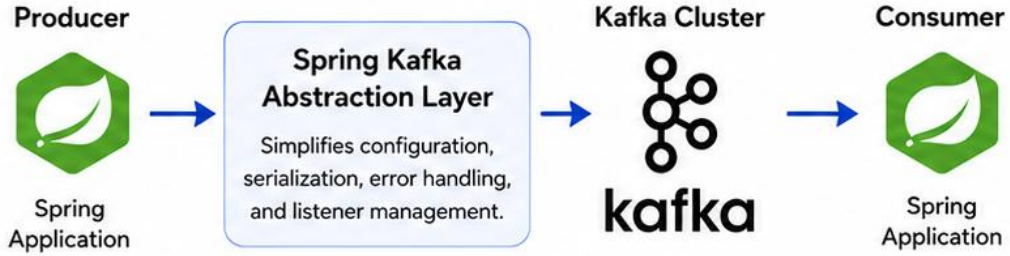
Seamless Integration of Apache Kafka with the Spring Ecosystem

## What is Spring Kafka?

Spring Kafka is a part of the Spring framework that provides easy integration and simplified configuration for building Kafka-based messaging applications.

spring Kafka

## How It Works




## Key Benefits

- Simplified configuration with Spring Boot
- Annotation-driven programming model
- Built-in support for retries, error handling, and dead letter topics
- Seamless integration with Spring ecosystem
- Highly testable and production-ready

## Core Components

**ProducerTemplate**
Simplifies sending messages to Kafka topics.

**@KafkaListener**
Annotation to consume messages using listener containers.

**Consumer Factory & Producer Factory**
Configure and create consumer and producer instances.

**Error Handling**
Built-in mechanisms for retries, recovery, and dead letter topics.

**Embedded Kafka (for Testing)**
Supports integration testing with embedded Kafka broker.

## Typical Use Cases

- E-Commerce Order & Event Processing
- Real-time Analytics & Monitoring
- Notification & Email Services
- Data Synchronization Between Systems
- Microservices Communication via Events

**Bottom Line**

Spring Kafka helps developers build robust, scalable, and event-driven applications by simplifying Kafka integration while leveraging the power of the Spring framework.

# KAFKA INTEGRATION ARCHITECTURE

*Spring Boot applications use Spring Kafka to produce and consume events from Apache Kafka, enabling scalable, reliable, and decoupled systems.*

## KEY COMPONENTS

- **Producers —** Spring Boot services publish events using KafkaTemplate.
- **Kafka Cluster —** a distributed, fault-tolerant platform that stores and streams events.
- **Consumers —** Spring Boot services consume events using @KafkaListener.
- **Serialization —** events are serialized (JSON, Avro) for interoperability.
- **External Systems —** databases, caches, or services that interact with producers/consumers.

## END-TO-END FLOW

- 1. Producer service creates an event.
- 2. Event is published to a Kafka topic.
- 3. Kafka brokers store and replicate the event.
- 4. Consumer service(s) read the event.
- 5. Event is processed and action is taken.

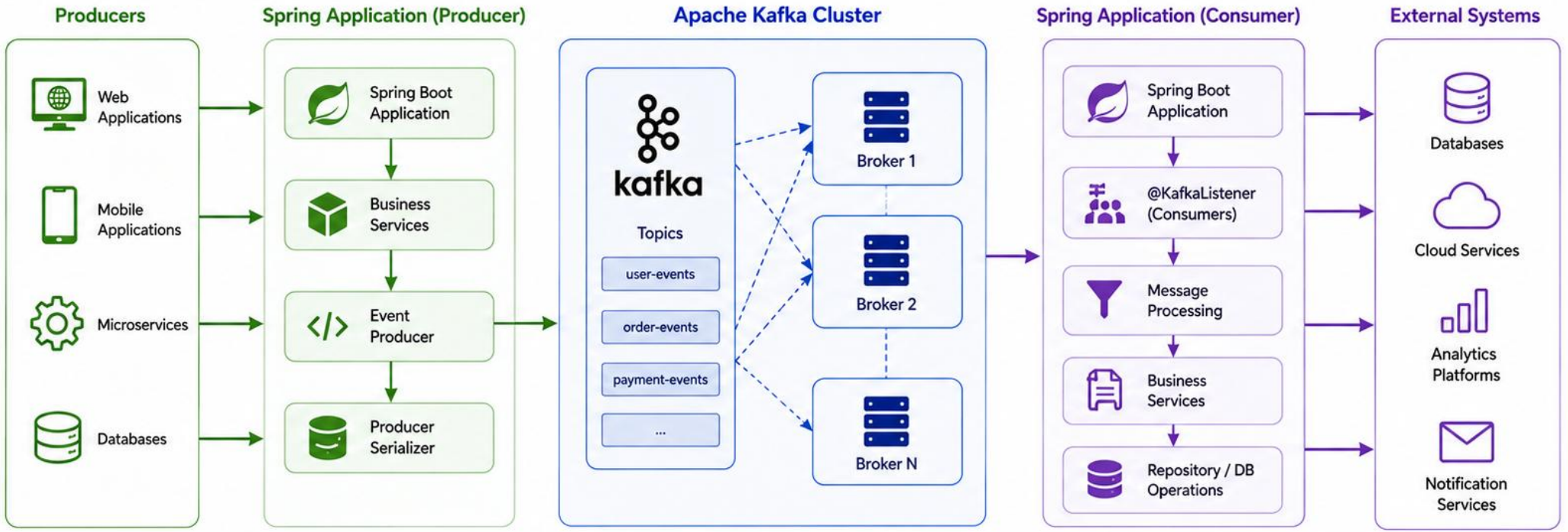
## BENEFITS

- **Decoupled —** services communicate asynchronously via events.
- **Scalable —** producers and consumers scale independently.
- **Reliable —** durable storage and replication ensure no data loss.
- **High Throughput —** Kafka handles millions of events per second.

In Northstar CRM: the CustomerController/Service persists the customer, then CustomerEventPublisher (KafkaTemplate) publishes to crm.customer-events.v1 -- consumed by CustomerEventListener.

**KEY TAKEAWAY** Spring Boot services publish and consume events through Kafka, enabling scalable, reliable, and decoupled systems across the enterprise application landscape.

# Kafka Integration Architecture




**Cross-Cutting Concerns**

- Security (SASL/SSL)
- Monitoring & Metrics
- Error Handling & Retries
- Dead Letter Topics
- Configuration Management

**Benefits**

- Loose Coupling
- Scalability & High Throughput
- Real-time Streaming
- Reliability & Fault Tolerance
- Data Consistency & Insights

# SPRING KAFKA COMPONENTS

*Spring Kafka provides a set of core components and abstractions that simplify building reliable and scalable Kafka-based applications.*

## CORE COMPONENTS

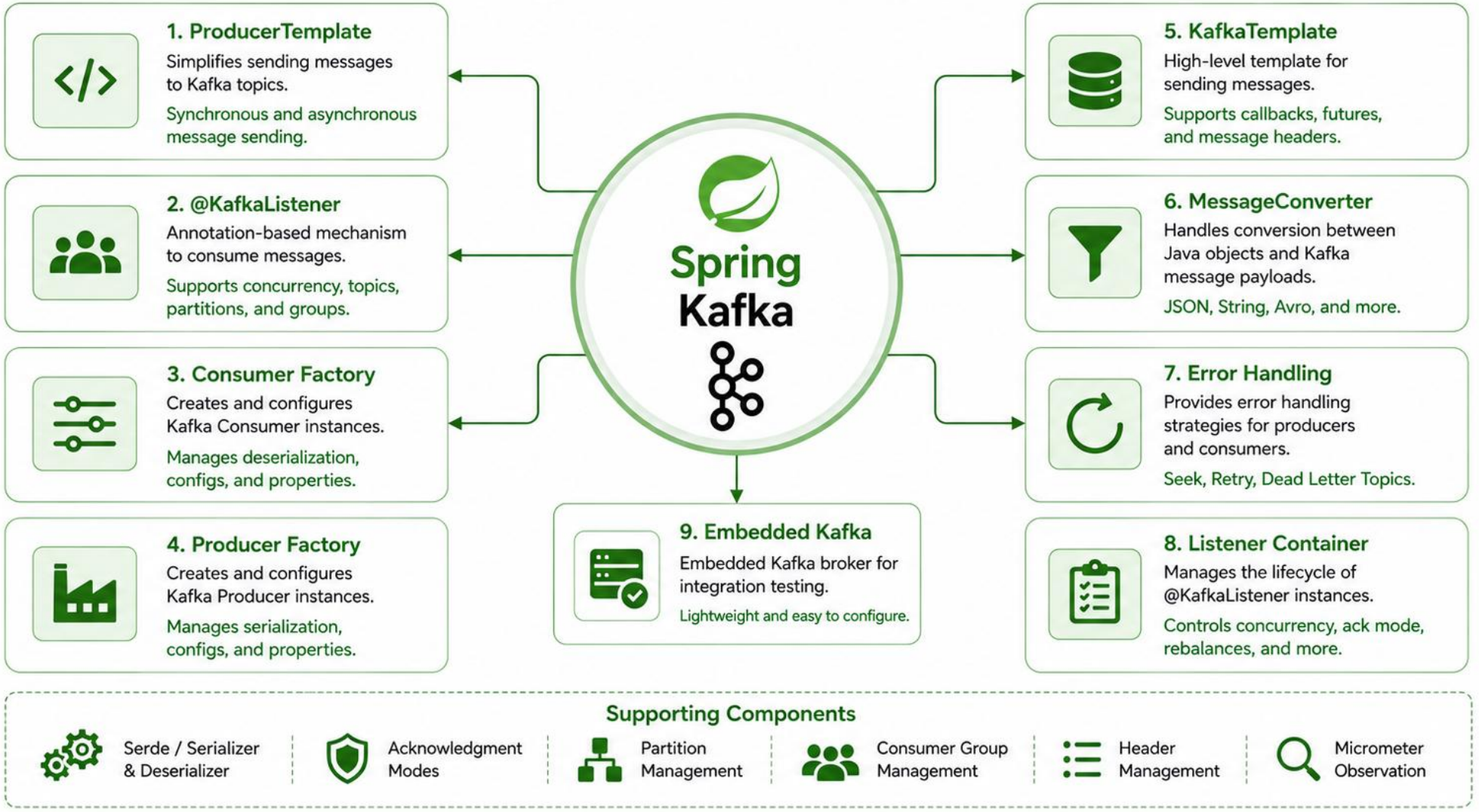
| Component | Role |
|---|---|
| **KafkaTemplate** | High-level abstraction for producing messages to Kafka topics. |
| **@KafkaListener** | Annotation-based programming model for consuming messages. |
| **Consumer Group** | Multiple consumers working together to consume messages from topics. |
| **ConsumerFactory / ProducerFactory** | Factories that create configured Kafka producer and consumer instances. |
| **Serialization / Deserialization** | Converts objects to/from bytes using Key/Value Serializer and Deserializer. |
| **KafkaProperties** | Holds Kafka configuration properties from application.properties/yml. |
| **MessageConverter** | Converts messages between Java objects and Kafka record payloads. |
| **Error Handling Components** | Provides error handling, retries, and Dead Letter Queue (DLQ) support. |

## SUPPORTING COMPONENTS

**Ack Mode** | **Offset Management** | **Message Key** | **RecordInterceptor** | **Metrics & Observation**

**KEY TAKEAWAY** Spring Kafka components provide high-level abstractions, reduce boilerplate code, and help you build robust, scalable, and maintainable event-driven applications with ease.

# Spring Kafka Components



# TRY IT YOURSELF — EXERCISE 1: SPRING KAFKA ROLES

*Reinforces: mapping Kafka concepts to Spring Boot components -- KafkaTemplate, @KafkaListener, bootstrap-servers, group-id.*

## STEP-BY-STEP

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Connect KafkaTemplate and @KafkaListener to producer/consumer concepts. |
| **Step 1** | Study the reference mapping (right) and copy it into notes/lab31-spring-kafka.md. |
| **Step 2** | CRM story: after HTTP creates Amina, the service calls KafkaTemplate to publish to crm.customer-events.v1 keyed by CUS-1001. |
| **Step 3** | Listener story: the notifications listener uses group crm-notifications and processes the JSON envelope. |
| **Step 4** | Gap check: list one open question about serializers (String vs. JSON) before lab. |
| **Deliverable** | Role-mapping notes + CRM produce/consume story + one serializer question (table copied, both stories written, question listed). |

## REFERENCE: KAFKA IDEA -> SPRING PIECE

```
// Kafka idea -> Spring Boot piece
Produce record      -> KafkaTemplate.send(...)
Consume record       -> @KafkaListener
Bootstrap servers    -> spring.kafka.bootstrap-servers
Group id              -> spring.kafka.consumer.group-id

// CRM story (notes/lab31-spring-kafka.md)
produce: HTTP creates Amina -> KafkaTemplate
         .send("crm.customer-events.v1",
               "CUS-1001", event)
listen:  group=crm-notifications processes
         the JSON envelope
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-spring-kafka-roles.md.

# KAFKA PRODUCER ARCHITECTURE

*A Kafka producer application publishes events to topics in a Kafka cluster. Spring Kafka simplifies producer development and configuration.*

## PRODUCER FLOW

- **1. Create Message —** the application creates a domain event or data payload.
- **2. Send via KafkaTemplate —** KafkaTemplate sends the message with optional key, headers, and partition.
- **3. Serialization —** KeySerializer and ValueSerializer convert objects to bytes.
- **4. Partitioner —** the message is assigned to a partition based on key or strategy.
- **5. Broker Acknowledgement —** the message is sent to the leader broker and acknowledged per the acks policy.

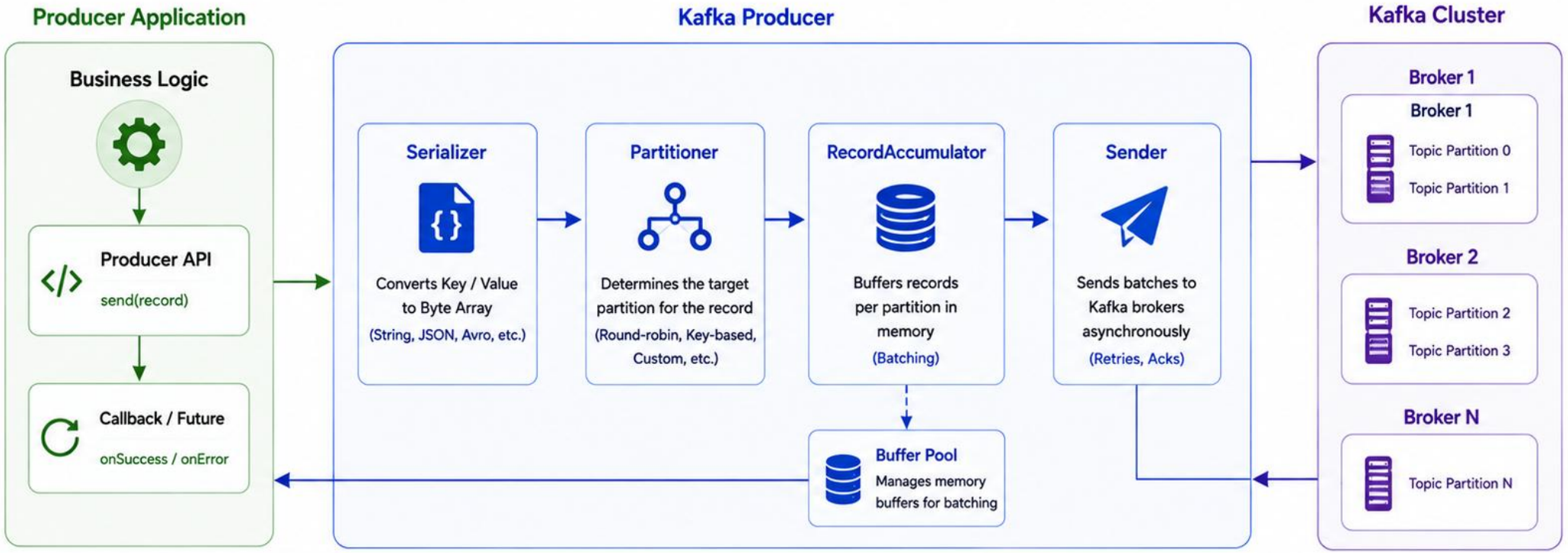
## PRODUCER CONFIGURATION (application.yml)

```
spring:
  kafka:
    bootstrap-servers: ${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
    producer:
      properties:
        enable.idempotence: true
        acks: all
        retries: 3
```

| acks | Description | Durability |
|---|---|---|
| **0** | No acknowledgement | Low |
| **1** | Leader acknowledges | Medium |
| **all (-1)** | All in-sync replicas acknowledge | High |

**KEY TAKEAWAY** Spring Kafka and KafkaTemplate work together to efficiently publish messages to Kafka topics with serialization, batching, partitioning, and fault-tolerant delivery.

# Kafka Producer Architecture




## Key Features

- Asynchronous & Non-blocking
- Batching for High Throughput
- Reliable Delivery (Retries & Acks)
- Configurable & Flexible
- Extensible (Serializer, Partitioner, etc.)

## Producer Configurations

- bootstrap.servers
- key.serializer
- value.serializer
- acks
- retries
- batch.size
- linger.ms
- compression.type

# KAFKATEMPLATE OVERVIEW

*KafkaTemplate is the central class in Spring Kafka for sending messages. It provides a simplified, thread-safe API over the native Kafka Producer.*

## KEY CHARACTERISTICS

- Thread-safe -- one KafkaTemplate bean can be shared across the whole application.
- Uses configured serializers, interceptors, and producer properties automatically.
- Supports both synchronous (blocking) and asynchronous (non-blocking) sends.
- Returns a CompletableFuture carrying send metadata (partition, offset) for async handling.
- Easily configured through standard Spring Boot application properties.

## LAB 31: CustomerEventPublisher.java

```
@Service
public class CustomerEventPublisher {
  private final KafkaTemplate<String, CustomerEvent> template;

  public void publish(String topic, CustomerEvent event) {
    template.send(topic, event.customerId(), event)
      .whenComplete((result, error) -> {
        if (error != null)
          log.error("customer_event_publish_failed id={}",
              event.eventId(), error);
        else
          log.info("customer_event_published id={} offset={}",
              event.eventId(), result.getRecordMetadata().offset());
      });
  }
}
```

**KEY TAKEAWAY** KafkaTemplate is the recommended way to publish messages in Spring Kafka. It abstracts the complexity of the native Kafka Producer while providing reliable, efficient event publishing.

# KafkaTemplate Overview

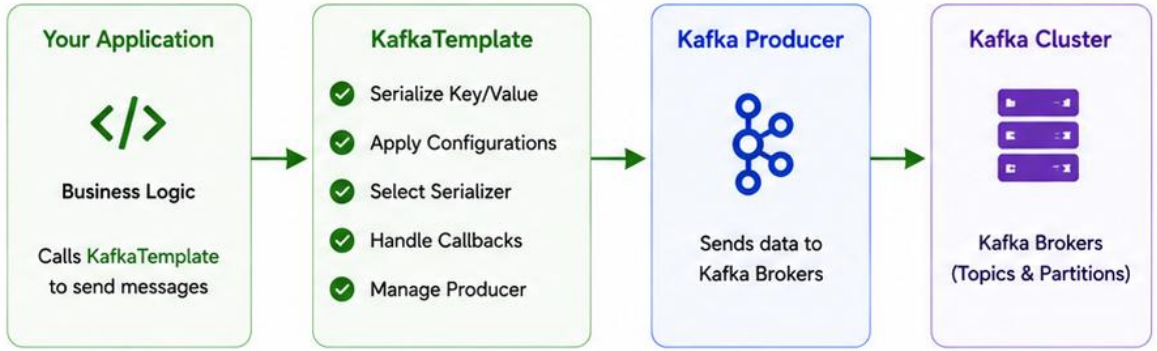
High-level abstraction for producing messages in Spring Kafka

## What is KafkaTemplate?

KafkaTemplate is a high-level abstraction provided by Spring Kafka to simplify sending messages to Kafka topics.

It wraps the Kafka Producer API and provides a thread-safe, feature-rich interface.

## How KafkaTemplate Works




## Key Features

**Simplified API**
Easy methods to send messages with less boilerplate code.

**Automatic Serialization**
Converts objects to byte array using configured serializers.

**Callback Support**
Supports success and failure callbacks for async operations.

**Producer Reuse**
Manages and reuses underlying Kafka Producer efficiently.

**Thread Safe**
Safe to use in multi-threaded environments.

## KafkaTemplate Send Flow


## Common Send Methods

**send(topic, data)**
Send data with auto-generated key

**send(topic, key, data)**
Send data with specified key

**send(ProducerRecord)**
Send using full ProducerRecord

**sendDefault(data)**
Send data to default topic

## Benefits

- Developer Productivity
- Reduced Boilerplate
- Consistent Configuration
- Reliable & Robust

# PUBLISHING MESSAGES

*Publishing messages means sending data from a Spring Boot application to a Kafka topic so that other services can consume and act on it.*

## KEY POINTS

- Use KafkaTemplate to send messages -- never the raw KafkaProducer client directly.
- Messages are published to a named topic (crm.customer-events.v1 in Lab 31).
- Kafka handles durability, replication, and ordering within a partition.
- Publishing after a successful database write reflects a fact that already happened.
- Always handle the send result (success/failure); never publish and forget.

## COMMON SEND OPTIONS

| Option | Purpose |
|---|---|
| **Topic** | The destination topic where the message is published. |
| **Key** | Used for partitioning and ordering -- customerId in Lab 31. |
| **Value (payload)** | The actual message content -- the CustomerEvent. |
| **Headers** | Metadata about the message (e.g. correlationId, eventType). |
| **Partition** | Let Kafka decide by key hash (default), or specify one explicitly. |

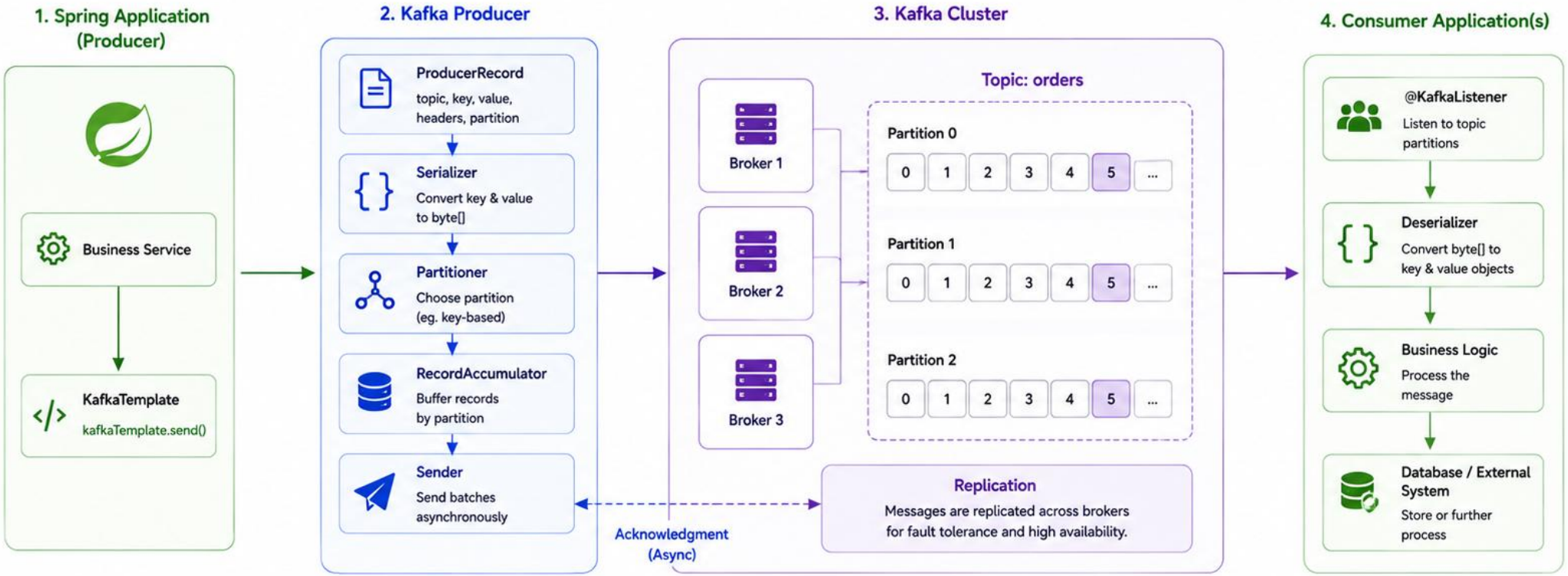
Wire the publisher into the customer create/status-change service methods so events are published only after the CRM database write succeeds -- never speculatively before it.

**Use meaningful keys** **Keep payloads small** **Handle send results** **Prefer async sends**

**KEY TAKEAWAY** Publishing messages moves a fact from your application into Kafka's durable, replicated log so that other services can consume and act on it independently.

# Publishing Messages

From Spring Application to Kafka Topic




# SERIALIZING EVENTS

*Serialization converts Java objects into a byte array so they can be transmitted over the network and stored in Kafka.*

## WHY SERIALIZATION MATTERS

- **Network Communication —** Kafka brokers store and transfer data as bytes.
- **Interoperability —** different services and languages can read the same data.
- **Efficiency —** compact formats reduce payload size and network overhead.
- **Schema Evolution —** controlled changes to the event shape without breaking consumers.

| Serializer | Use Case |
|---|---|
| **StringSerializer** | For simple text messages / message keys. |
| **JsonSerializer** | For JSON payloads -- recommended for microservices (Lab 31). |
| **ByteArraySerializer** | For raw byte[] messages. |

## SPRING KAFKA CONFIGURATION (Lab 31)

```
spring:
  kafka:
    consumer:
      group-id: crm-notifications
      auto-offset-reset: earliest
      enable-auto-commit: false
      properties:
        spring.json.trusted.packages: com.northstar.crm.event
    producer:
      properties:
        enable.idempotence: true
```

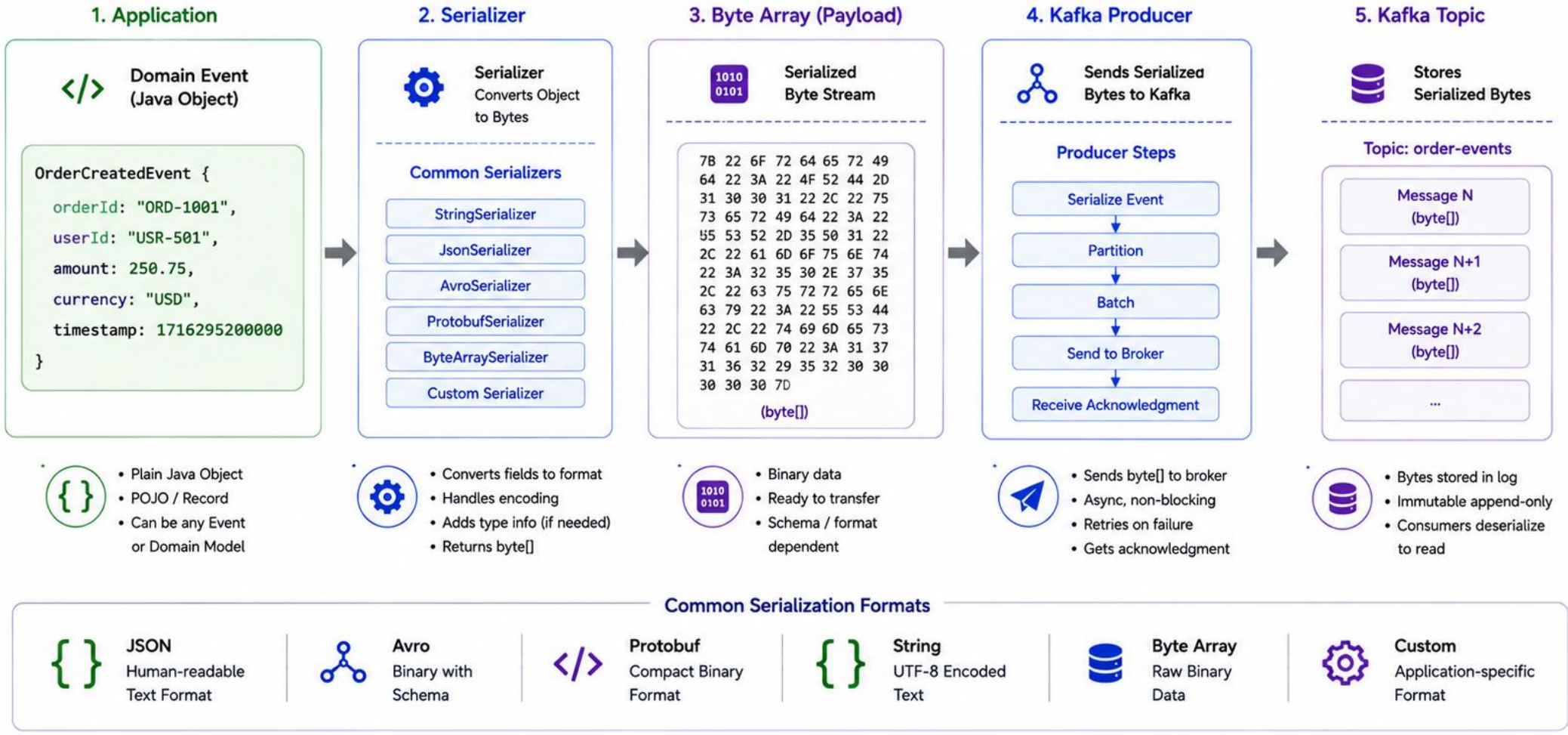
**Restrict trusted packages** **Keep contracts stable**

**Version events (v1)** **Never expose internals**

**KEY TAKEAWAY** Serialization is the bridge between your application objects and Kafka's byte-based messaging system -- choose the right serializer, keep contracts stable, and publish meaningful, versioned events.

# Serializing Events

Convert Java Objects to Byte Streams before Publishing to Kafka




## Common Serialization Formats

| | | | | | |
|---|---|---|---|---|---|
| **JSON** Human-readable Text Format | **Avro** Binary with Schema | **Protobuf** Compact Binary Format | **String** UTF-8 Encoded Text | **Byte Array** Raw Binary Data | **Custom** Application-specific Format |

# KAFKA CONSUMER ARCHITECTURE

*A Kafka consumer application subscribes to topics, processes messages, and commits offsets to ensure reliable, at-least-once (or exactly-once) processing.*

## CONSUMER FLOW

1. **Subscribe**
2. **Partitions**
3. **Poll Records**
4. **Process**
5. **Commit**

## OFFSET COMMIT STRATEGIES

| Strategy | Description |
|---|---|
| **Auto Commit** | Offsets committed automatically at intervals (default). |
| **Manual Commit** | Application commits after successful processing (Lab 31). |
| **Commit Sync** | Blocks until the broker acknowledges the commit. |
| **Commit Async** | Commits in the background (non-blocking). |

## KEY CONSUMER CONCEPTS

- **Consumer Group —** enables multiple instances to share the work (crm-notifications in Lab 31).
- **Partitions —** messages in a topic are split across partitions for parallelism.
- **Offset —** a unique, monotonically increasing position within a partition.
- **At-Least-Once —** the default delivery guarantee; exactly-once needs idempotent design.

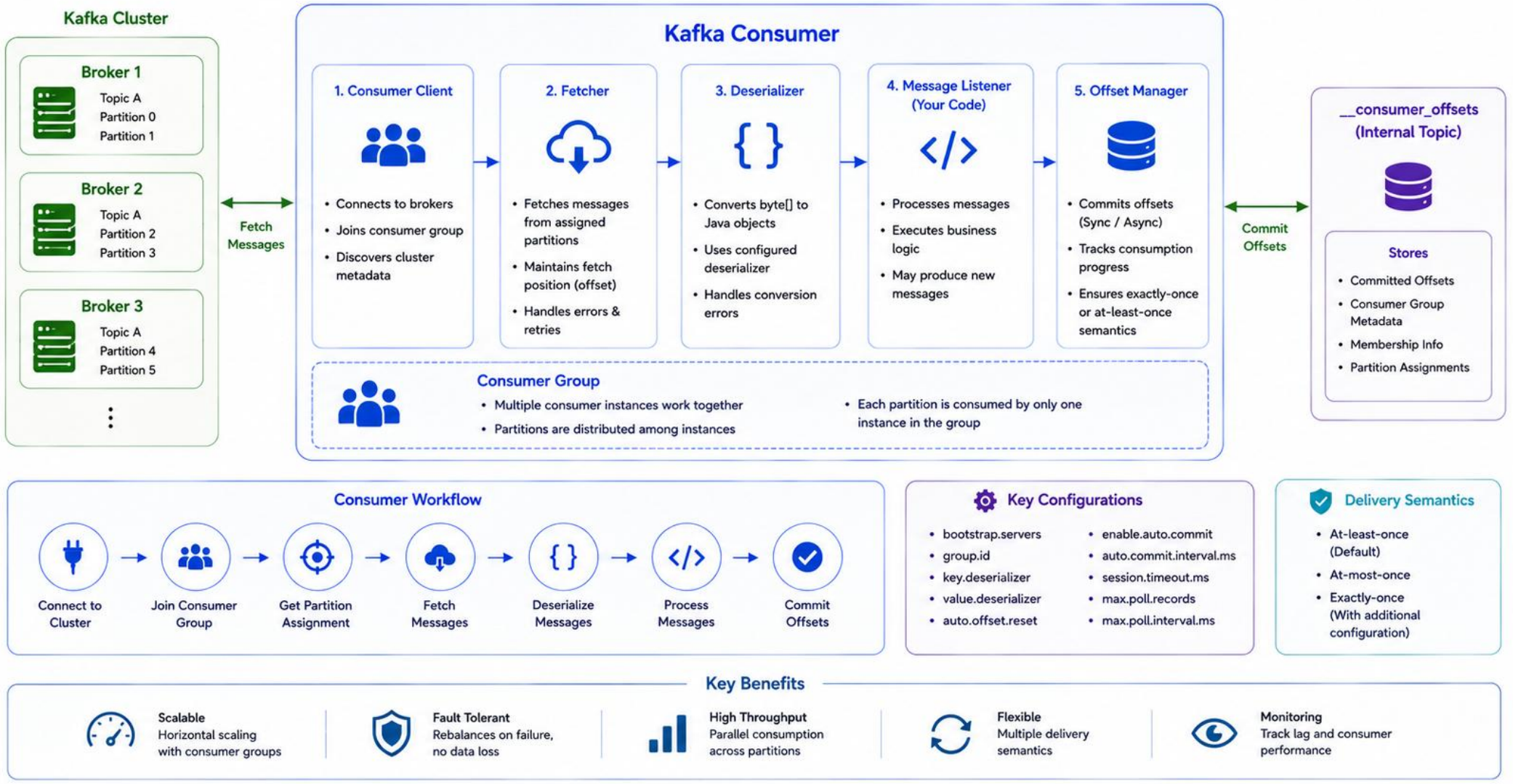
## Lab 31 consumer config

```
group-id: crm-notifications
enable-auto-commit: false
auto-offset-reset: earliest
```

Scalable | Reliable | Flexible | Resilient

**KEY TAKEAWAY** Kafka consumers enable your applications to reliably subscribe, process, and track progress of messages. Proper offset management ensures data is not lost and processing is reliable.

# Kafka Consumer Architecture



# @KAFKALISTENER ANNOTATION

*@KafkaListener is the core annotation in Spring Kafka used to declare a method that listens to one or more topics and processes incoming messages.*

## KEY FEATURES

- Simplifies consumer configuration -- Spring creates the consumer container for you.
- Supports topic subscription, consumer group, partitions, and concurrency.
- Supports single record, batch, or manual-acknowledgment processing modes.
- Integrates with error handling, retries, and Dead Letter Topics.
- Flexible method signatures -- payload, full record, or specific headers.

| Attribute | Purpose |
|---|---|
| **topics** | Topics to subscribe to (property-driven in Lab 31). |
| **groupId** | Consumer group id (crm-notifications). |
| **@Header(...)** | Access a specific Kafka header, e.g. RECEIVED_KEY. |

## LAB 31: CustomerEventListener.java

```
@Component
public class CustomerEventListener {

  @KafkaListener(topics = "${crm.kafka.customer-topic}")
  void receive(CustomerEvent event,
      @Header(KafkaHeaders.RECEIVED_KEY) String key) {

    if (!key.equals(event.customerId()))
      throw new InvalidCustomerEventException("key mismatch");

    log.info("customer_event_received id={} customerId={}",
        event.eventId(), event.customerId());

    handler.handle(event);
  }
}
```

**KEY TAKEAWAY** @KafkaListener declares a method to be a Kafka message listener -- Spring automatically creates the consumer container and invokes the method for each received record.

# @KafkaListener Annotation

Simplifies consumer configuration by using annotations in Spring Kafka

## What is @KafkaListener?

@KafkaListener is a Spring Kafka annotation that marks a method to listen to one or more Kafka topics and process incoming messages.

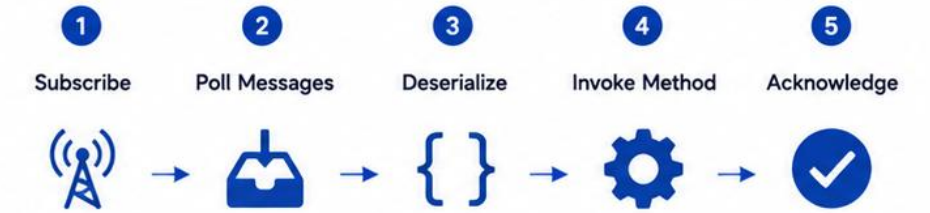
## Basic Usage

```
@KafkaListener(topics = "orders",
    groupId = "order-service-group")
public void listen(OrderCreatedEvent event) {
    // process event
}
```

## Key Attributes

| | |
|---|---|
| topics | Topic name(s) to subscribe |
| groupId | Consumer group ID |
| topicPartitions | Specify topic and partition(s) |
| containerFactory | Reference to listener container factory |
| clientIdPrefix | Prefix for consumer client ID |
| autoStartup | Start listener container automatically |
| concurrency | Number of concurrent consumers |

## How @KafkaListener Works




## Listener Container


## Example

```
@Component
public class OrderEventListener {

    @KafkaListener(
      topics = "orders",
      groupId = "order-service-group",
      containerFactory = "kafkaListenerContainerFactory",
      concurrency = "3")
    )
    public void handle(OrderCreatedEvent event,
                Acknowledgment ack) {
       // business logic
       processOrder(event);
        ack.acknowledge(); // manual ack
    }
}
```

## Supported Method Signatures

- void listen(String message)
- void listen(MyEvent event)
- void listen(ConsumerRecord<K, V> record)
- void listen(MyEvent event, Acknowledgment ack)
- void listen(MyEvent event, @Header("key") String key)
- ListenableFuture<?> listen(MyEvent event)

## Benefits

- Simplifies configuration with annotation-based declaration
- Less boilerplate code
- Supports multiple topics & patterns
- Built-in container management
- Robust error handling & retry support
- Scalable with concurrency

# TRY IT YOURSELF — EXERCISE 2: LISTENER SKETCH

*Reinforces: @KafkaListener method signatures, distinct consumer groups on the same topic, and payload typing.*

## STEP-BY-STEP

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Sketch two listeners (notifications vs. audit) without compiling code. |
| **Step 1** | Method outline: on paper, write @KafkaListener(topics="crm.customer-events.v1", groupId="crm-notifications") void onCustomerEvent(...). |
| **Step 2** | Second group: sketch the audit listener with groupId "crm-audit" on the same topic. |
| **Step 3** | Payload type: choose String/JsonNode or a typed CustomerEvent DTO -- justify the choice in one line. |
| **Step 4** | Correlation: note where correlationId ("lab-request-001") gets logged for support. |
| **Deliverable** | Two sketched listeners, distinct group IDs, same topic, plus a typing decision (both groupIds present, same topic, typing + correlation notes written). |

## SKETCH: TWO @KAFKALISTENER METHODS

```
// Notifications listener (from CustomerEventListener)
@KafkaListener(
    topics = "crm.customer-events.v1",
    groupId = "crm-notifications")
void onCustomerEvent(CustomerEvent event) { ... }

// Exercise 2: sketch the audit listener
@KafkaListener(
    topics = "crm.customer-events.v1",
    groupId = "crm-audit")
void onCustomerEventForAudit(CustomerEvent e) { ... }

// payload type: typed CustomerEvent DTO -- justify in 1
line
// log correlationId ("lab-request-001") for support
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-listener-sketch.md.

# CONSUMING EVENTS

*Consuming events means reading messages from Kafka topics and processing them reliably, while managing offsets and errors correctly.*

## CONSUMER MODES (DELIVERY SEMANTICS)

| Mode | Behavior |
|---|---|
| **At-Least-Once (default)** | No message is lost; duplicates may occur -- Lab 31's model. |
| **At-Most-Once** | No duplicates, but messages can be lost. |
| **Exactly-Once (EOS)** | No duplicates, no loss -- requires idempotent producer + transactions. |

## BEST PRACTICES

- **Meaningful groupId**
- **Right offset strategy**
- **Idempotent processing**
- **Monitor consumer lag**
- **Retries + DLT**

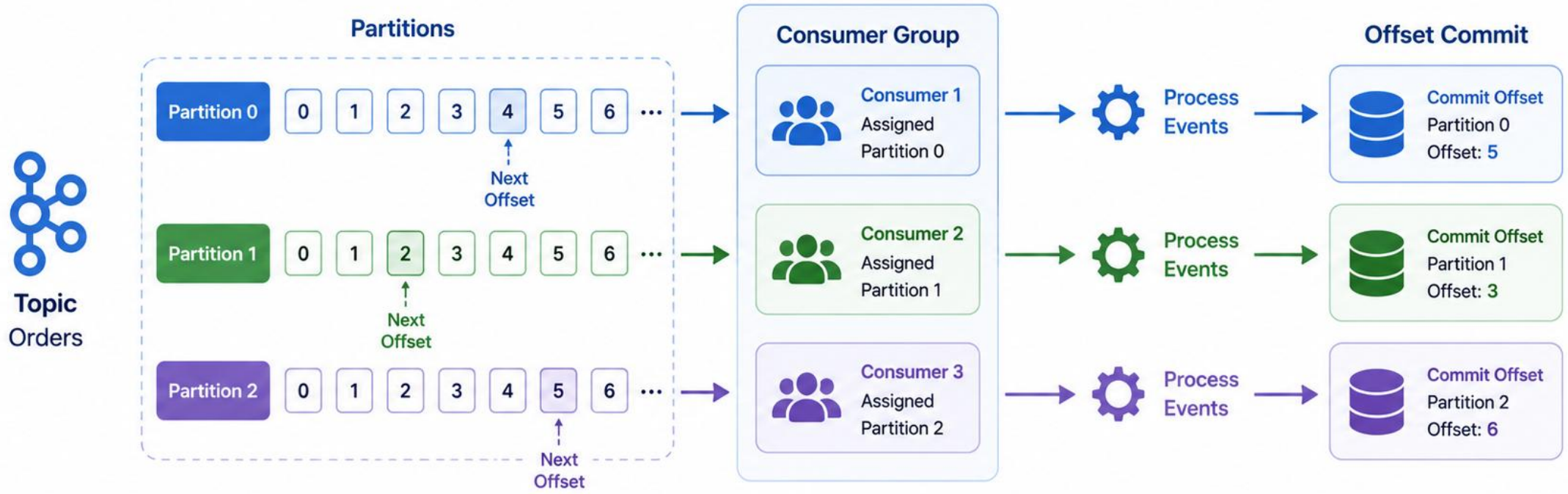
## EXAMPLE (@KafkaListener) -- OrderConsumer.java (general pattern)

```
@Service
public class OrderConsumer {

  @KafkaListener(topics = "orders",
      groupId = "order-service-group",
      containerFactory = "kafkaListenerContainerFactory")
  public void listen(OrderEvent event,
      Acknowledgment ack) {
    // Business logic
    process(event);

    // Manual offset commit only after success
    ack.acknowledge();
  }
}
```

**KEY TAKEAWAY** Consuming events reliably -- at-least-once or exactly-once -- requires the right offset strategy and idempotent, well-monitored processing logic.

# Consuming Events

Consumers read events from topics, process them, and commit offsets to track progress.




**1 Poll**
Consumer polls records from assigned partitions starting at the last committed offset.

**2 Receive Events**
Broker returns available events for each assigned partition.

**3 Process Events**
Consumer processes the events (business logic, transformations, etc.).

**4 Commit Offset**
After successful processing, consumer commits the latest offset.

**5 Track Progress**
Committed offsets are stored by the broker to track progress and enable recovery.

# CONSUMER GROUPS IN SPRING

*A consumer group is a set of consumers that work together to consume data from topics. Kafka ensures each partition is read by only one consumer in a group.*

## WHY CONSUMER GROUPS?

- **Scalability —** add more consumer instances to increase throughput.
- **Reliability —** if one consumer fails, others in the group continue.
- **Load Balancing —** partitions are distributed among consumers automatically.
- **Independent Groups —** different groups (e.g. crm-notifications, crm-audit) can read the same topic independently.

## EXAMPLE: @KAFKALISTENER WITH GROUP ID

```
@KafkaListener(
    topics = "${crm.kafka.customer-topic}",
    groupId = "crm-notifications")
void receive(CustomerEvent event) {
  // process event once per group
}
```

| Scenario | What Happens |
|---|---|
| **More consumers than partitions** | Some consumers stay idle. |
| **A consumer leaves or fails** | Its partitions rebalance to the rest of the group. |

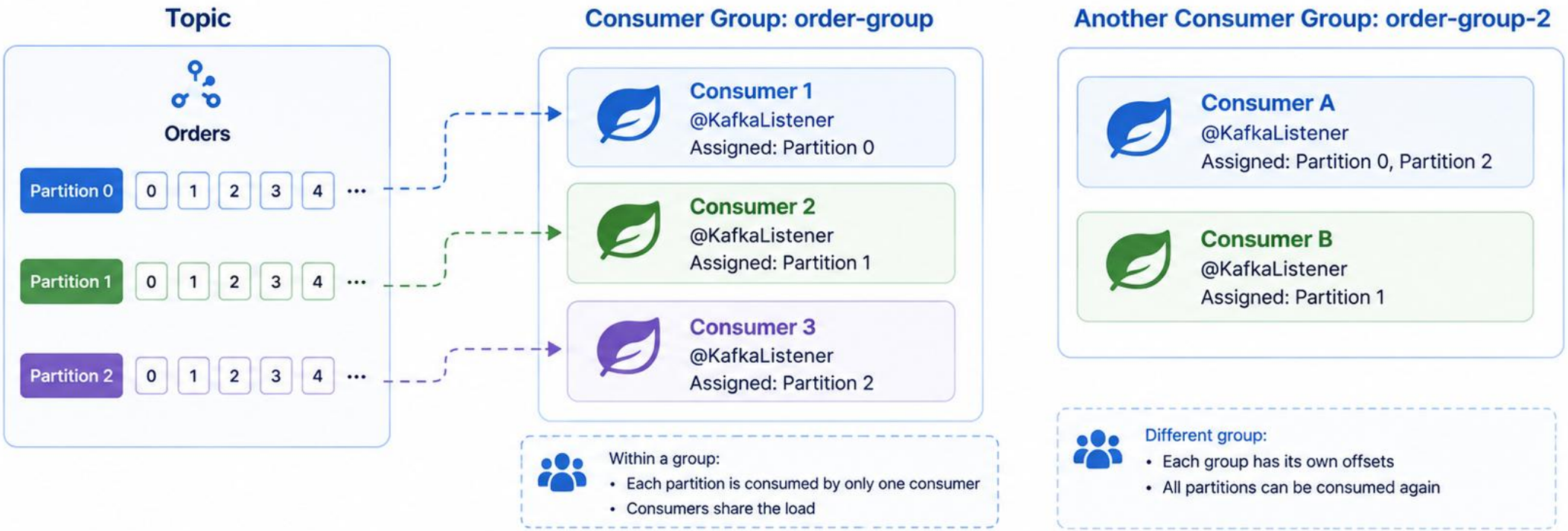
## BEST PRACTICES

**Meaningful groupId** | **Size group to partitions** | **Enable retries** | **Monitor rebalances**

**KEY TAKEAWAY** Consumer groups enable scalable, reliable, and parallel processing. Understanding partition assignment and rebalancing is key to building robust event-driven applications.

# Consumer Groups in Spring

A consumer group is a set of consumers that work together to consume messages from topics. Each partition is consumed by only one consumer in the group.




## Key Points

- Consumers in the same group coordinate and divide partitions.
- Offsets are committed per group, not globally.
- If a consumer fails, another in the group takes over its partitions.
- In Spring, use **@KafkaListener** with group-id.

# TRY IT YOURSELF — EXERCISE 3: FILL SPRING KAFKA TODOS (STARTER)

*Reinforces: basic Spring Kafka producer/consumer wiring -- bootstrap-servers, group-id, topic strings, and message keys.*

## STEP-BY-STEP

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Fill the TODOs in a tiny Spring Kafka pseudocode snippet -- a STARTER, not a finished solution. |
| **Step 1** | Paste the snippet (right) into notes/lab31-todos.md. |
| **Step 2** | Fill blanks with: localhost:9092 (or the instructor's bootstrap address), crm-notifications, and crm.customer-events.v1 (used twice). |
| **Step 3** | Key reminder: add a comment that the key argument must be CUS-1001 / CUS-1002, never a random UUID. |
| **Step 4** | DLT blank: add // TODO Lab 31: route poison messages to crm.customer-events.v1.dlq. |
| **Deliverable** | Filled pseudocode with topic/bootstrap/group replaced and key/DLT reminders (all _____ replaced, customer ID key comment present, DLT TODO line present). |

## STARTER SNIPPET -- FILL EVERY BLANK

```
// application.yml ideas
spring.kafka.bootstrap-servers: _____
spring.kafka.consumer.group-id: _____

@Service
class CustomerEventPublisher {
  private final KafkaTemplate<String, String> template;
  void publishCreated(String customerId, String json) {
    template.send("_____", customerId, json); // topic
  }
}

@KafkaListener(topics = "_____", groupId = "crm-
notifications")
void onEvent(String payload) { /* TODO: parse +
idempotent handle */ }

// TODO Lab 31: route poison messages to
//   crm.customer-events.v1.dlq
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-fill-spring-kafka-todos.md.

# MESSAGE KEYS

*A message key is an optional piece of data sent with each record. It determines how the record is stored in Kafka and how ordering is maintained.*

## WHY USE MESSAGE KEYS?

- **Ordering Guarantee —** records with the same key are always written to, and consumed from, the same partition in order.
- **Data Locality —** related events (same key) stay together in the same partition.
- **Idempotency Support —** keys help implement idempotent writes and deduplication.
- **Better Parallelism —** different keys spread across partitions for higher throughput.

| Key Considerations | |
|---|---|
| **null key** | Kafka uses round-robin partitioning -- no ordering guarantee. |
| **low-cardinality key** | Too few distinct keys causes uneven load (a partition hot-spot). |

## LAB 31: PUBLISHING WITH A KEY

```
// CUS-1001 is the key for every Amina Khan event
kafkaTemplate.send(
    "crm.customer-events.v1",
    event.customerId(),   // key = "CUS-1001"
    event);                // value = CustomerEvent
```

## COMMON KEY TYPES

| Key Type | Use Case |
|---|---|
| **String** | Customer ID, Order ID, User ID (Lab 31: customerId). |
| **UUID** | Unique correlation or event IDs. |

**KEY TAKEAWAY** Use message keys to control partitioning, maintain order, and keep related data together. Same key -> same partition -> ordered processing.

# Message Keys

## Control Partitioning, Ordering & Event Correlation

### 1. What is a Message Key?

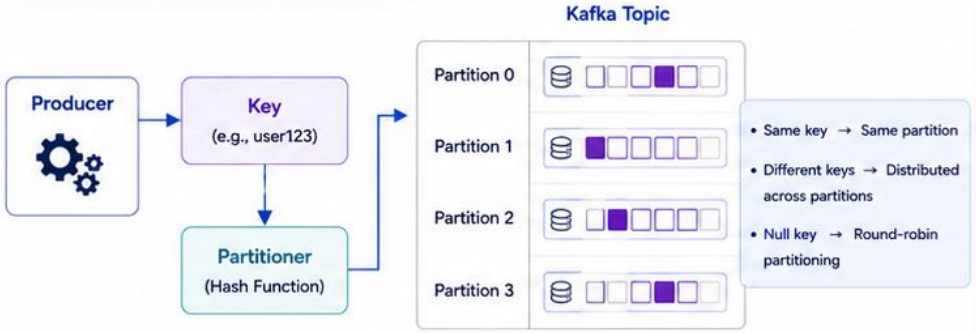
A message key is an optional attribute in a Kafka record.

Kafka uses the key (not the value) to determine which partition the message is stored in.

**Kafka Record**

| Key | Value |
|---|---|
| (e.g., userId) | (e.g., OrderCreatedEvent) |

### 2. How Keys Determine Partitions




### 3. Producer Example (Spring Kafka)

```
@Autowired
private KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

public void publishOrder(OrderCreatedEvent event) {
  String key = event.getUserId();  // message key

  kafkaTemplate.send(
       "order-events",       // topic
       key,                  // key
       event                 // value
  );
}
```

### 4. Why Keys Matter

**Ordering**
Messages with the same key are written to the same partition and consumed in order.

**Event Correlation**
All events related to the same entity (e.g., user, orderId) stay in the same partition.

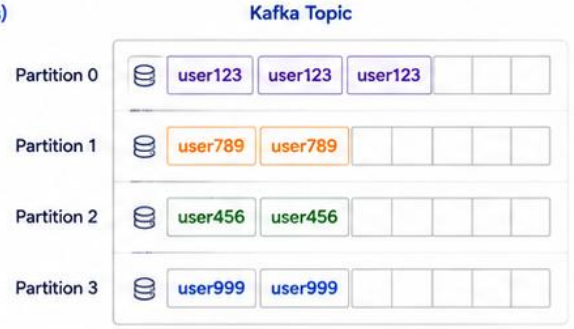
**Scalability**
Different keys are distributed across partitions for parallelism.

**Idempotency & Deduplication**
Keys help identify and handle duplicate events for the same entity.

### 5. Key Distribution Example

**Topic: order-events (4 partitions)**

| Message Key | Partition |
|---|---|
| user123 | Partition 0 |
| user456 | Partition 2 |
| user789 | Partition 1 |
| user123 | Partition 0 |
| user456 | Partition 2 |
| user999 | Partition 3 |




All messages with the same key go to the same partition.

### 6. Best Practices

- Use a key when order and correlation matter (e.g., userId, orderId).
- Choose high-cardinality keys to ensure even distribution.
- Avoid null keys unless ordering is not important.
- Keep key size small and consistent.
- Design keys that are stable and unique per entity.

### 7. Key Takeaway

Message keys are the foundation for ordering, correlation, and scalable event-driven systems in Kafka.

# OFFSET MANAGEMENT

*Offsets represent the position of a consumer in a partition. Managing offsets correctly ensures reliable processing, no data loss, and no duplicate processing.*

## WHAT IS AN OFFSET?

- A unique, monotonically increasing number identifying a record's position within a partition, starting from 0.
- Offsets are stored by Kafka itself, tracked per consumer group + topic + partition.

| Commit Strategy | Description |
|---|---|
| **Auto Commit** | Committed automatically at regular intervals (default). |
| **Manual Commit** | Application commits after successful processing (recommended). |
| **Commit Sync** | Blocks until the offset is successfully committed. |
| **Commit Async** | Commits in the background (non-blocking). |

## AUTO OFFSET RESET (no committed offset exists)

| Value | Description |
|---|---|
| **earliest** | Start reading from the earliest available offset (Lab 31's setting). |
| **latest** | Start reading from the latest offset (new records only). |
| **none** | Throw an exception if no committed offset is found. |

## BEST PRACTICES

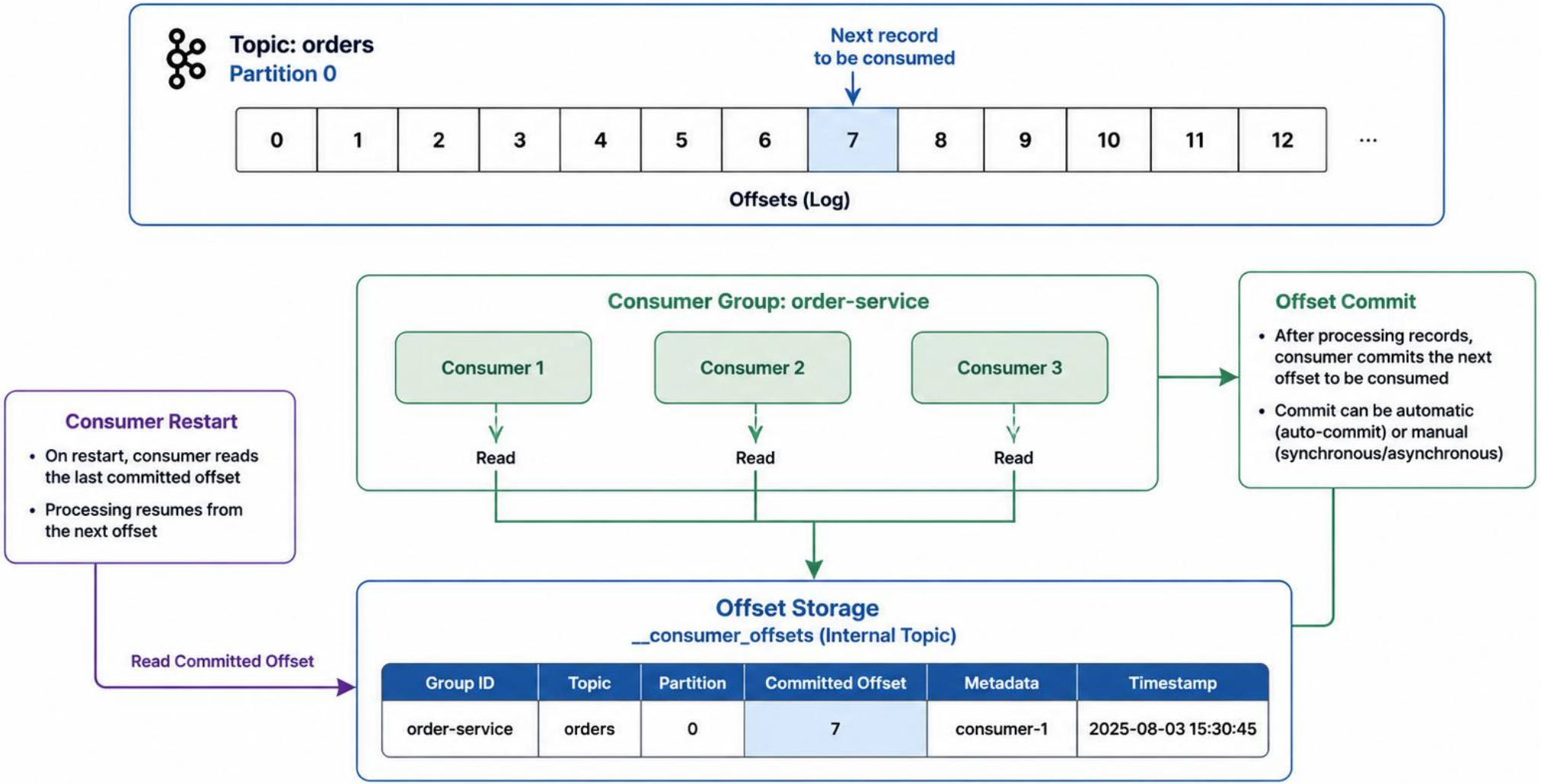
**Disable auto-commit** **Commit after success**

**Never commit on failure** **Monitor consumer lag**

Lab 31: enable-auto-commit: false and auto-offset-reset: earliest -- the application controls exactly when progress is recorded.

**KEY TAKEAWAY** Offsets = progress. Commit them at the right time -- after success, never on failure -- to achieve no data loss and no duplicates.

# Offset Management



# ERROR HANDLING STRATEGIES

*Robust error handling ensures message processing is resilient, failures are isolated, and messages are not lost. Spring Kafka provides multiple mechanisms to handle errors gracefully.*

## TYPES OF ERRORS

- **DeserializationException —** failed to convert message key/value (bad JSON, wrong trusted package).
- **Non-Retryable Contract Errors —** key mismatch, unsupported event version -- retrying can never fix these.
- **Transient Infrastructure Errors —** broker unavailable, timeout -- often self-resolve.

| Strategy | Use Case |
|---|---|
| **Retry** | Transient failures (timeouts, temporary unavailability). |
| **Skip** | Non-critical messages; bad data that can be ignored. |
| **Dead Letter Topic** | Permanent failures, poison messages (Lab 31's approach). |
| **Stop Consumer** | Critical, unrecoverable system issues. |

## LAB 31: KafkaErrorConfig.java

```
@Bean
public DefaultErrorHandler errorHandler(
    KafkaTemplate<Object, Object> template) {
  var recoverer = new DeadLetterPublishingRecoverer(template);
  var handler = new DefaultErrorHandler(
      recoverer, new FixedBackOff(1000, 2));

  handler.addNotRetryableExceptions(
      InvalidCustomerEventException.class,
      UnsupportedEventVersionException.class);
  return handler;
}
```

**Bounded retries** **Route poison to DLT** **Never infinite-retry**

**KEY TAKEAWAY** Choose the right strategy based on the type of error: retry transient failures with backoff, route permanent failures to a Dead Letter Topic, and never let a bad contract retry forever.

# Error Handling Strategies

### 1. Retry Mechanism

- Automatically retry failed operations
- Use exponential backoff
- Best for transient errors

**Use Case**
Temporary network issues, timeout

### 2. Dead Letter Queue (DLQ)

- Send failed messages to a DLQ
- Process or analyze later
- Prevents message loss

**Use Case**
Deserialization errors, invalid data

### 3. Skip & Log

- Log the error and skip the message
- Continue processing next messages
- Prevents pipeline blocking

**Use Case**
Non-critical messages, minor data issues

### 4. Alert & Notify

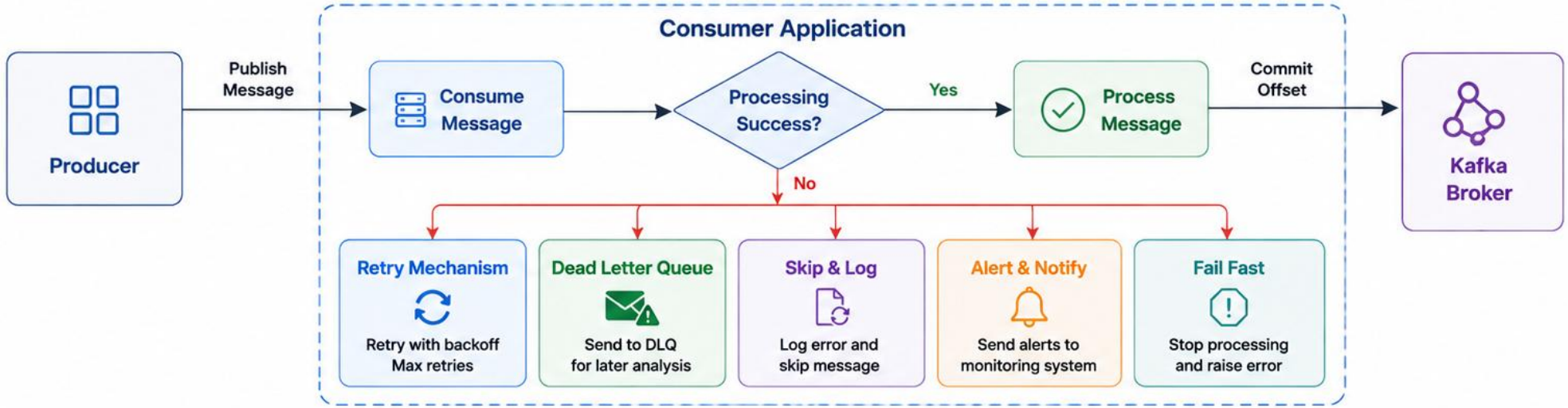
- Raise alerts for failures
- Notify operators or teams
- Requires monitoring setup

**Use Case**
Critical failures, system down

### 5. Fail Fast

- Stop processing immediately
- Raise error to trigger recovery
- Ensures data consistency

**Use Case**
Data corruption, security issues



# DEAD LETTER TOPICS (DLT)

*A Dead Letter Topic is a special Kafka topic where messages that cannot be processed successfully are sent, so they are not lost and can be inspected, retried, or fixed later.*

## WHAT'S PRESERVED IN A DLT RECORD

| Header | Example |
|---|---|
| **kafka_dlt-exception-fqcn** | InvalidCustomerEventException |
| **kafka_dlt-exception-message** | key mismatch |
| **kafka_dlt-original-topic** | crm.customer-events.v1 |
| **kafka_dlt-original-partition / -offset** | 1 / 15 |

## EXAMPLE: DLT CONSUMER

```
@KafkaListener(topics = "crm.customer-events.v1.DLT",
    groupId = "dlq-consumer-group")
public void listenDlt(ConsumerRecord<String, String> record) {
  log.warn("dlt_message key={} headers={}",
      record.key(), record.headers());
  // Inspect, alert, fix, or trigger reprocessing
}
```

## DLT NAMING -- LAB 31 REQUIREMENT

Lab 31 requires documenting which naming convention is used: Spring's default .DLT suffix, or Lab 30's .dlq suffix -- an undocumented choice makes .dlq consumers look 'empty' while Spring silently writes elsewhere.

**<topic>.DLT (Spring default)** **<topic>.dlq (Lab 30)**

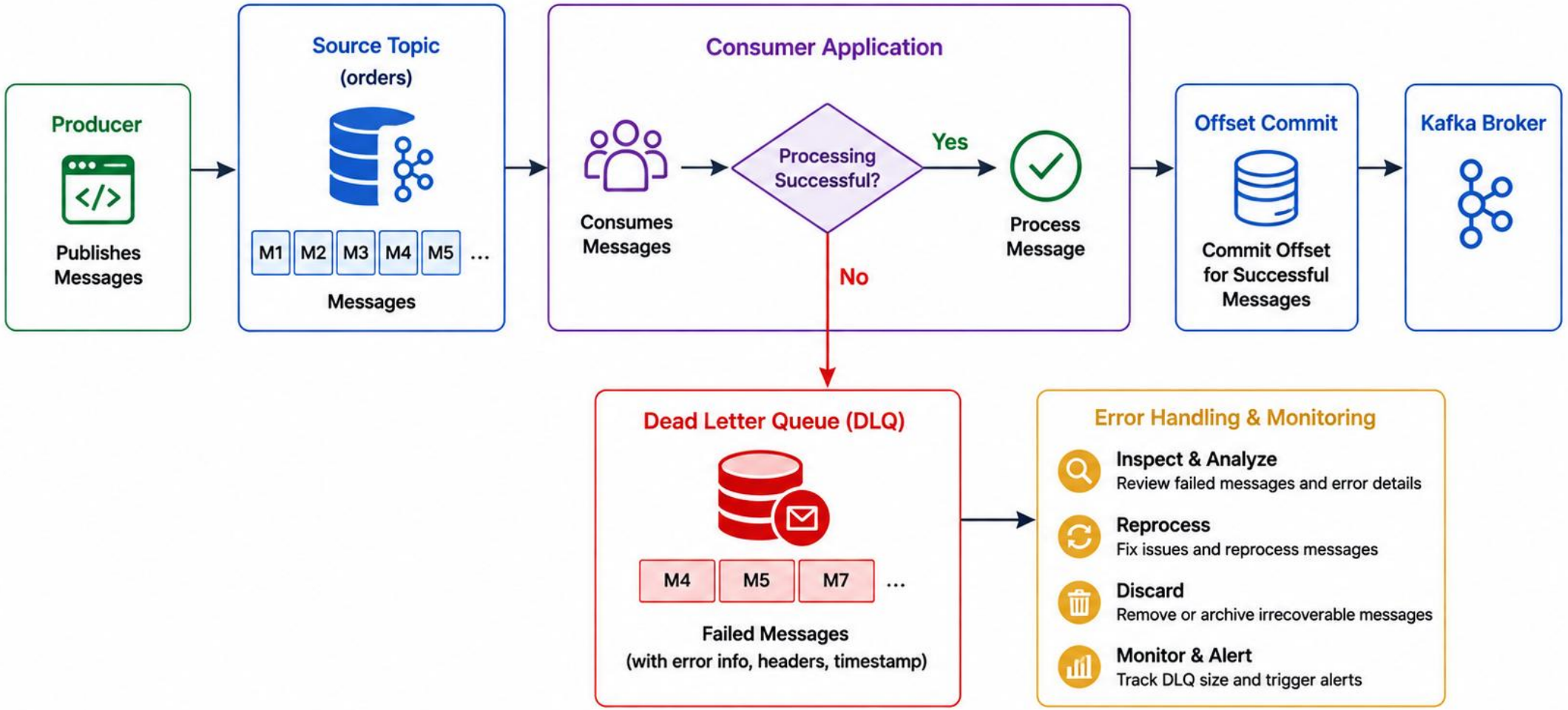
## BEST PRACTICES

**Monitor DLT size** **Alert on growth**

**Analyze & clean regularly** **Avoid infinite reprocess loops**

**KEY TAKEAWAY** A DLT prevents message loss and keeps the main topic flow healthy -- it preserves the original message, headers, and exception info for later inspection, retry, or reprocessing.

# Dead Letter Queues



# TRY IT YOURSELF — EXERCISE 4: ERROR AND DLT NOTES

*Reinforces: classifying retryable vs. non-retryable errors and documenting DLT replay ownership.*

## STEP-BY-STEP

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Describe when a listener should retry vs. send to the DLT. |
| **Step 1** | Retryable example: a transient network blip calling the email API -- retry. |
| **Step 2** | Non-retryable example: JSON missing customerId -- send to DLT after limited attempts. |
| **Step 3** | Ops note: support replays the DLT after fixing the consumer, using correlationId lab-request-001. |
| **Step 4** | No runtime: confirm you will NOT publish to the DLT from the CLI in this pre-lab. |
| **Deliverable** | Clear retry-vs-DLT decision notes for Lab 31 (retryable example, non-retryable example, replay/ops sentence). |

## NOTES: RETRY VS. DLT

```
// notes/lab31-error-dlt.md

Retryable:
  transient network blip calling email API
  -> retry

Non-retryable:
  JSON missing customerId -> DLT after
  limited attempts

Ops note:
  support replays DLT after fixing consumer,
  using correlationId "lab-request-001"

// No runtime: do NOT publish to DLT via CLI here
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-error-dlt-notes.md.

# RELIABLE EVENT PROCESSING (1 OF 2)

*Reliable event processing ensures messages are processed exactly once or at least once without loss or duplication, even in the presence of failures.*

## DELIVERY SEMANTICS

- **At-Least-Once (default) —** a message may be processed more than once, but will never be lost. Achieved by committing offsets after processing.
- **Exactly-Once (end-to-end) —** a message is processed once and only once. Requires idempotent producer + transactions + atomic offset commit.

## RELIABLE EVENT PROCESSING FLOW


Commit offsets only AFTER the side effects are successfully completed -- committing first risks silently losing the event if a crash happens mid-processing.

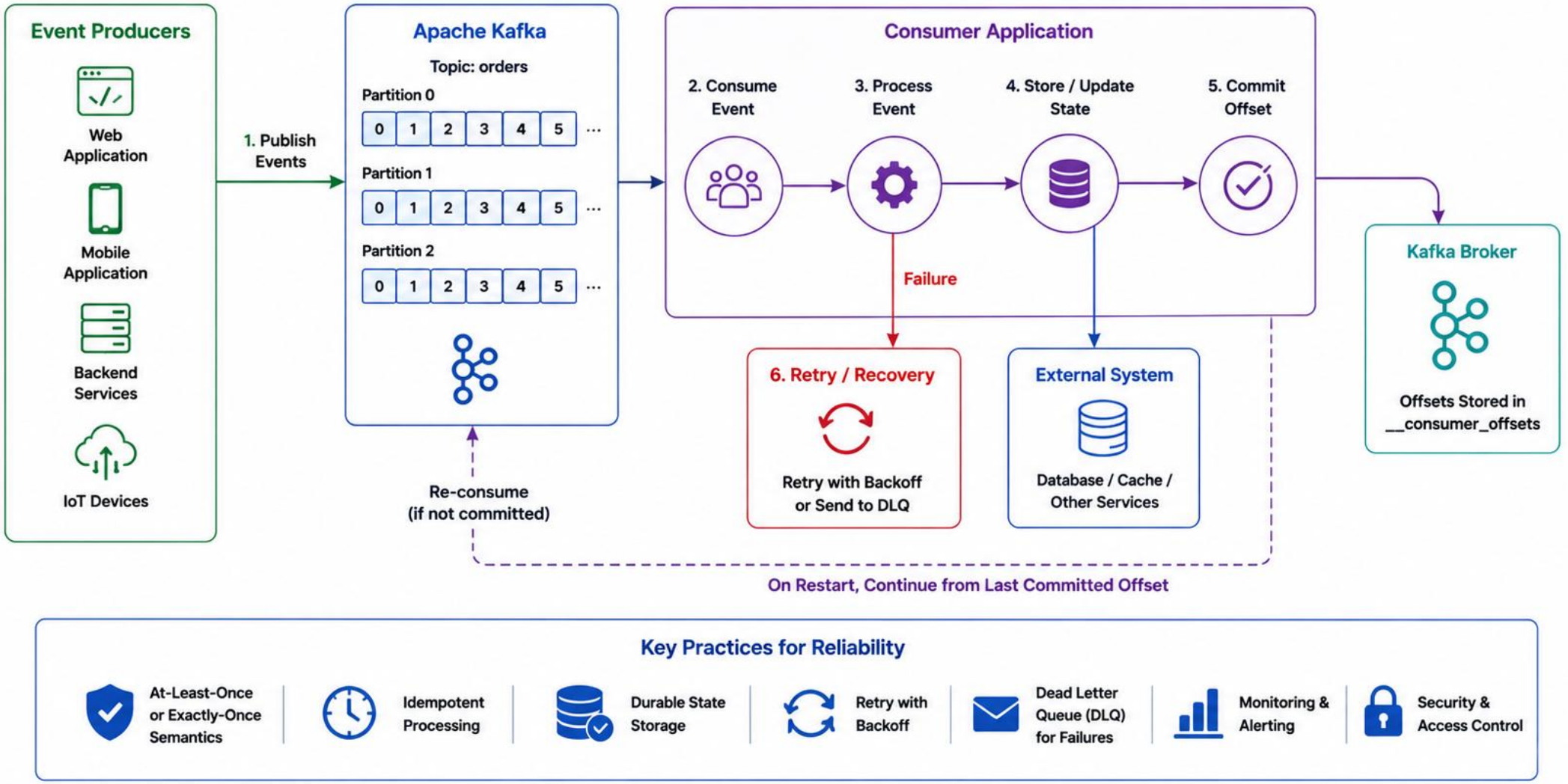
## KEY RELIABILITY PRINCIPLES

- **Durability —** Kafka replicates data across brokers to prevent loss.
- **Idempotency —** processing the same event twice has the same result.
- **Atomicity —** business logic + offset commit happen as one unit.
- **Consistency —** the system stays valid even after failures.

| Scenario | Impact | Handling |
|---|---|---|
| **Consumer crashes** | Event may be reprocessed | At-least-once + idempotent logic |
| **Network failure** | May cause duplicates | Idempotent producer + consumer |

**KEY TAKEAWAY** Reliable event processing is a combination of Kafka's durability guarantees and application design -- idempotency and atomicity, not Kafka configuration alone.

# Reliable Event Processing




## Key Practices for Reliability

- At-Least-Once or Exactly-Once Semantics
- Idempotent Processing
- Durable State Storage
- Retry with Backoff
- Dead Letter Queue (DLQ) for Failures
- Monitoring & Alerting
- Security & Access Control

# RELIABLE EVENT PROCESSING (2 OF 2)

*Idempotent consumer design is what turns Kafka's at-least-once guarantee into correct, once-only business behavior -- built and proven in Lab 31.*

## LAB 31: ProcessedEventStore.java

```
public class ProcessedEventStore {
  private final Set<UUID> processedIds =
      ConcurrentHashMap.newKeySet();

  public boolean markIfNew(UUID eventId) {
    return processedIds.add(eventId);
  }
}

// in the listener:
if (!store.markIfNew(event.eventId())) {
  log.info("duplicate_event_ignored id={}", event.eventId());
  return;
}
notificationService.notify(event);
```

Mark BEFORE the side effect, not after -- marking after risks a duplicate notification if the process crashes between the side effect and the mark.

## COMMON FAILURE SCENARIOS & HANDLING

| Scenario | Impact | Strategy |
|---|---|---|
| **Duplicate events (replay/rebalance)** | Duplicate side effects | Unique eventId + ProcessedEventStore |
| **DB failure after processing** | Offset not committed | Commit only after successful update |
| **Broker failure** | Temporary unavailability | Replication + retries ensure no loss |

## BEST PRACTICES

- **Commit after success only**
- **Idempotent producers**
- **Design consumers to be idempotent**
- **Monitor lag & DLT**

Production note: an in-memory Set resets on restart -- production systems use a durable, shared unique-key table (database) instead.

**KEY TAKEAWAY** Idempotent consumer design (mark-then-notify, never notify-then-mark) is what proves once-only business effects even though Kafka itself only guarantees at-least-once delivery.

# TRY IT YOURSELF — EXERCISE 5: IDEMPOTENCY PLAN

*Reinforces: designing an idempotency key and processed-event check for duplicate-safe consumers, on paper.*

## STEP-BY-STEP

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Define how a consumer ignores a second delivery of Amina's Created event. |
| **Step 1** | Why duplicates: list two causes -- producer retry, consumer rebalance/reprocess. |
| **Step 2** | Business key: propose an idempotency key, e.g. eventId or customerId+eventType+occurredAt, for CUS-1001. |
| **Step 3** | Store idea: one sentence -- check a processed-events table/set before side effects (email). |
| **Step 4** | Out of scope: do not implement the table yet -- paper design only. |
| **Deliverable** | A short idempotency plan tied to Northstar customer events (two duplicate causes, concrete key proposal, processed-store idea stated). |

## SKETCH: IDEMPOTENCY NOTES

```
// notes/lab31-idempotency-plan.md

Duplicate cause 1: producer retry
Duplicate cause 2: consumer rebalance / reprocess

Idempotency key:
  customerId + eventType + occurredAt
  (e.g. "CUS-1001")

Store check (before side effect):
  if (!store.markIfNew(eventId)) return;
  // then, and only then: send notification
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-idempotency-plan.md.

# KAFKA BEST PRACTICES (1 OF 2)

*Following best practices helps build reliable, scalable, secure, and maintainable event-driven systems using Apache Kafka.*

| Category | Practices |
|---|---|
| **1. Topic Design** | Use meaningful, business-focused topic names. Keep topics focused on one event type. Choose an appropriate partition count for parallelism. Plan for scalability -- partitions can be increased but never decreased. |
| **2. Producer Practices** | Use acks=all for strong delivery guarantees. Enable compression (gzip/snappy/lz4) to reduce network usage. Use meaningful keys with good cardinality. Set timeouts (linger.ms, batch.size) properly. Monitor producer metrics. |
| **3. Consumer Practices** | Use consumer groups for parallel processing and availability. Commit offsets after successful processing (manual preferred). Handle failures gracefully with retries and a DLT. Tune fetch settings (max.poll.records) for the workload. |
| **4. Message Design** | Use message keys to control ordering and partitioning. Use structured formats (JSON, Avro/Protobuf with a schema registry). Include metadata (timestamps, schema version, correlationId). Make messages immutable -- publish new events for changes. |

**KEY TAKEAWAY** Design topics, producers, consumers, and message contracts deliberately -- these four foundations determine everything about how reliable and scalable your Kafka integration will be.

# Kafka Best Practices

## 1. Topic Design

- Use meaningful topic names
- Choose the right number of partitions
- Avoid too many small topics
- Use compaction for stateful data

## 2. Producer Best Practices

- Use acks=all for durability
- Enable idempotence
- Set appropriate batch size and linger.ms
- Handle exceptions and retries
- Use compression

## 3. Consumer Best Practices

- Use consumer groups
- Commit offsets correctly (avoid auto-commit)
- Use at-least-once or exactly-once semantics
- Handle rebalances properly
- Tune max.poll.records and session timeout

## 4. Performance Tuning

- Monitor throughput, latency and lag
- Tune partitions for parallelism
- Optimize batch size and compression
- Use appropriate retention policies
- Balance producers and consumers

## 5. Reliability & Durability

- Replicate data across brokers
- Use acks=all and min.insync.replicas
- Monitor ISR (In-Sync Replicas)
- Test backups and disaster recovery

## 6. Monitoring & Alerting

- Monitor broker, topic and consumer metrics
- Track consumer lag
- Set up alerts for failures, lag and resource usage
- Use tools like Prometheus, Grafana, Kafka UI

## 7. Security

- Enable TLS for encryption
- Use SASL for authentication
- Apply ACLs for access control
- Rotate credentials regularly
- Audit access and config changes

## 8. Schema Management

- Use Schema Registry
- Enforce compatibility rules
- Version your schemas
- Validate schema before producing data
- Avoid breaking changes

## 9. Error Handling

- Implement retries with backoff
- Use Dead Letter Queue (DLQ) for failures
- Log and monitor errors
- Design idempotent consumers
- Handle poison pills carefully

## 10. Operations

- Keep Kafka and clients up to date
- Test changes in staging environment
- Automate deployments and configs
- Document configurations and runbooks
- Regularly review and clean up unused topics

# KAFKA BEST PRACTICES (2 OF 2)

*Reliability, security, performance, operations, and development practices for production-ready Kafka integrations.*

| Category | Practices |
|---|---|
| **5. Reliability & Fault Tolerance** | Use adequate replication (RF >= 3 for production). Enable idempotent producers (enable.idempotence=true). Use transactions for exactly-once when truly required. Send failed records to a DLT -- never lose bad data. Use exponential backoff with bounded retries. |
| **6. Security** | Enable authentication (SASL/OAUTHBEARER) for clients and brokers. Enable authorization (ACLs) to control access to topics and groups. Encrypt data with TLS/SSL in transit and at rest. |
| **7. Performance & Scalability** | Right-size partitions -- more partitions means more parallelism, but avoid too many small ones. Use fast disks (SSD), sufficient CPU/memory/network. Monitor broker metrics and scale horizontally when needed. |
| **8. Operations & Monitoring** | Monitor broker health, consumer lag, throughput, error rates, and disk usage. Set up alerts for high lag, disk usage, and under-replicated partitions. Plan capacity for data growth. Perform regular maintenance and version updates. |
| **9. Development Practices** | Externalize configuration using Spring Profiles and environment variables. Test thoroughly with realistic data and failure scenarios. Use Docker Compose / Testcontainers for local development and testing. |

**KEY TAKEAWAY** Design for reliability, build for scale, secure by default, and monitor continuously -- these are the operating principles that keep Kafka production-ready over time.

# TRY IT YOURSELF — EXERCISE 6: LAB 31 READINESS

*Reinforces: self-checking Lab 30 dependencies, tooling, and expected evidence before starting Lab 31.*

## STEP-BY-STEP

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Confirm Lab 30 topic names are ready inputs for Spring Kafka wiring. |
| **Step 1** | Dependency: write that Lab 31 assumes crm.customer-events.v1 (+ DLQ) naming from Lab 30. |
| **Step 2** | Maven habit: note you will use JDK 21 + Maven with the spring-kafka dependency in the starter (not yet). |
| **Step 3** | Evidence preview: list evidence expected later -- a log line for Amina's event, listener metrics/notes. |
| **Step 4** | Pass mark: pass if Exercises 2-4 notes exist; otherwise revisit the TODOs. |
| **Deliverable** | A readiness note linking Lab 30 topics to the Spring Kafka lab goals (topic dependency stated, JDK 21/Maven note, pass/fail self-mark). |

## NOTES: LAB 31 READINESS

```
// notes/lab31-readiness.md

Dependency: crm.customer-events.v1 (+ DLQ)
            naming comes from Lab 30

Tooling: JDK 21 + Maven; spring-kafka
         dependency added in the lab starter

Evidence expected: log line for Amina's event,
                   listener metrics/notes

Pass mark: Exercises 2-4 notes exist,
           else revisit the TODOs
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-lab31-readiness.md.

# LAB OVERVIEW -- SPRING BOOT INTEGRATION WITH KAFKA

*Lab 31: Spring Boot Integration with Kafka -- Northstar CRM Listeners*

## BUSINESS SCENARIO

- Lab 30 proved the broker. Leadership now requires the CRM service itself to emit and consume versioned customer events -- without losing notifications on poison messages or double-processing replays.
- When an agent creates Amina (CUS-1001) or activates Ravi (CUS-1002), notification and audit processes must learn asynchronously; duplicate delivery must not send duplicate SMS/email side effects.
- Leadership freezes the rule: publish keyed CustomerEvent v1 after successful writes; consume idempotently; recover poison messages to a DLT after bounded retry.

## LEARNING OBJECTIVES

- Add Spring Kafka dependencies to the CRM service.
- Configure JSON producers/consumers with externalized topic names.
- Publish typed customer events with KafkaTemplate.
- Write @KafkaListener handlers validating key, version, and metadata.
- Configure bounded retries and dead-letter publishing.
- Write idempotent event-processing logic; test with Embedded Kafka.

## WHAT YOU BUILD

- Copy lab29-crm to lab31-crm; add spring-kafka (+test); define immutable CustomerEvent; publish with KafkaTemplate keyed by customerId; write @KafkaListener validating key<->payload; add ProcessedEventStore idempotency; configure DefaultErrorHandler + DeadLetterPublishingRecoverer with non-retryable contract errors; write an integration test that awaits a handled event.

**KEY TAKEAWAY** Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; topic crm.customer-events.v1, group crm-notifications, and correlation ID lab-request-001 appear throughout.

# LAB TASKS AND DELIVERABLES

*Northstar CRM Listeners -- implementation steps, deliverables, and the evaluation rubric.*

| # | Implementation Step |
|---|---|
| 1 | Branch CRM to lab31-crm; add spring-kafka (+ spring-kafka-test). |
| 2 | Configure JSON messaging: bootstrap servers, group id, trusted packages, topic name -- all externalized. |
| 3 | Define CustomerEvent record (eventId, eventType, eventVersion, customerId, correlationId, data) with null/version guards. |
| 4 | Publish with KafkaTemplate after successful writes; log publish success/failure with partition/offset. |
| 5 | Write a typed @KafkaListener validating key == customerId; log received event with correlationId. |
| 6 | Make processing idempotent: ProcessedEventStore.markIfNew(eventId) before notifying. |
| 7 | Configure DefaultErrorHandler + DeadLetterPublishingRecoverer with FixedBackOff and non-retryable exceptions. |
| 8 | Integration test with EmbeddedKafka/Testcontainers awaiting a specific eventId; document DLT naming + runbook. |

## EXPECTED DELIVERABLES

- Spring Kafka publisher + listener for CRM events.
- Idempotent processing evidence.
- Retry + DLT configuration with proof.
- Integration test output (EmbeddedKafka/Testcontainers).
- Successful Amina/Ravi path evidence.
- Controlled poison-message / duplicate evidence.
- Runbook + DLT naming notes; no secrets committed.

## RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

**KEY TAKEAWAY** Listener without idempotency -> incomplete. Infinite retry on bad contracts -> honor issue. Sleep-only tests -> quality marks lost (per the guide's own explicit notes).

# MODULE 31 SUMMARY

*In this module, we learned how to integrate Apache Kafka with Spring Boot to build reliable, scalable, event-driven microservices.*

## WHAT WE COVERED

### Consumer Groups

Enable load balancing and scalability across consumers.

### Message Keys

Ensure ordering and control partitioning strategy.

### Offset Management

Auto-commit, manual commit, and offset reset strategies.

### Dead Letter Queues

Capture and isolate failed messages for retry or analysis.

### Reliable Processing

Achieve at-least-once and exactly-once semantics.

### Kafka Best Practices

Design topics, producers, and consumers for production.

## MODULE FLOW RECAP

1. **KafkaTemplate Publish**
2. **@KafkaListener Consume**
3. **Idempotent Processing**
4. **Retry + DLT**
5. **Lab: Northstar CRM**

**KEY TAKEAWAY** Effective Kafka integration is essential for building enterprise-grade, event-driven APIs. It ensures scalability, reliability, and resilient, production-ready event processing.

# KNOWLEDGE CHECK (1 OF 2)

*Test your understanding of Spring Kafka components, consumer groups, message keys, and offset behavior.*

**1. Which Kafka component stores messages durably?**

**A.** Zookeeper
**B. Broker**
**C.** Producer
**D.** Consumer

**2. What is the primary purpose of a Consumer Group?**

**A.** To increase message retention
**B. To enable parallel consumption**
**C.** To replicate messages
**D.** To encrypt messages

**3. What does a message key help achieve?**

**A.** Reduce message size
**B. Ensure ordering and partitioning**
**C.** Increase throughput
**D.** Enable compression

**4. What happens when a consumer tries to read a topic with no committed offset available?**

**A.** It always reads from the latest offset
**B.** It always throws an error
**C. It starts from earliest or latest, based on auto-offset-reset**
**D.** It always replays from the beginning

**KEY TAKEAWAY** Brokers store messages durably; consumer groups enable parallel processing; keys guarantee ordering; auto-offset-reset controls behavior when no committed offset exists.

# KNOWLEDGE CHECK (2 OF 2)

*Four more questions on delivery guarantees, Dead Letter Queues, producer acknowledgments, and testing tools.*

**5. Which delivery guarantee ensures that messages are processed exactly once?**

**A.** At-most-once

**B.** At-least-once

**C. Exactly-once**

**D.** Best-effort once

**6. What is the purpose of a Dead Letter Queue (DLQ)?**

**A.** To increase topic partitions

**B.** To store successfully processed messages

**C. To capture failed messages for retry or analysis**

**D.** To speed up consumers

**7. Which producer setting ensures a message is written to all in-sync replicas?**

**A.** acks=0

**B.** acks=1

**C. acks=all**

**D.** acks=-1 (a distinct setting from acks=all)

**8. Which tool is used in this module's lab to test Kafka-integrated REST APIs?**

**A. Postman**

**B.** Maven

**C.** JUnit

**D.** Docker

**KEY TAKEAWAY** Exactly-once needs idempotent producers + transactions; a DLQ captures failed messages for later analysis; acks=all waits for every in-sync replica; Postman exercises the REST layer around Kafka.

*"Publish the fact. Consume it once. Never lose the message."*

MODULE 31 COMPLETE

# Kafka Integration with Spring Boot

You can now publish events with KafkaTemplate, consume them reliably with @KafkaListener, build idempotent processing, and recover failures with retries and Dead Letter Topics.